

Fragmented Architectures

Patterns with smaller components

Contents

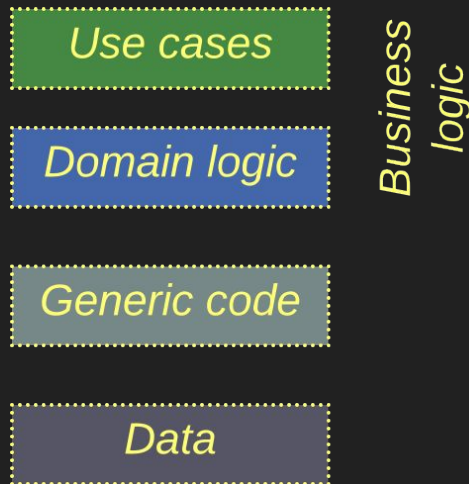
Several patterns don't rely on system-wide layers:

- *Layered Services* – divide into services, then into layers.
- *Polyglot Persistence* – employ multiple databases.
- *Backends for Frontends (BFF)* – dedicate a service to each kind of client.
- *Service-Oriented Architecture (SOA)* – divide into layers, then into services.
- *Hierarchy* – recursive subdivision.

Schema

Some of the patterns we'll cover define the primary architecture of the system.

Others extend a basic architecture, such as subdomain *Services* or *Layers*.



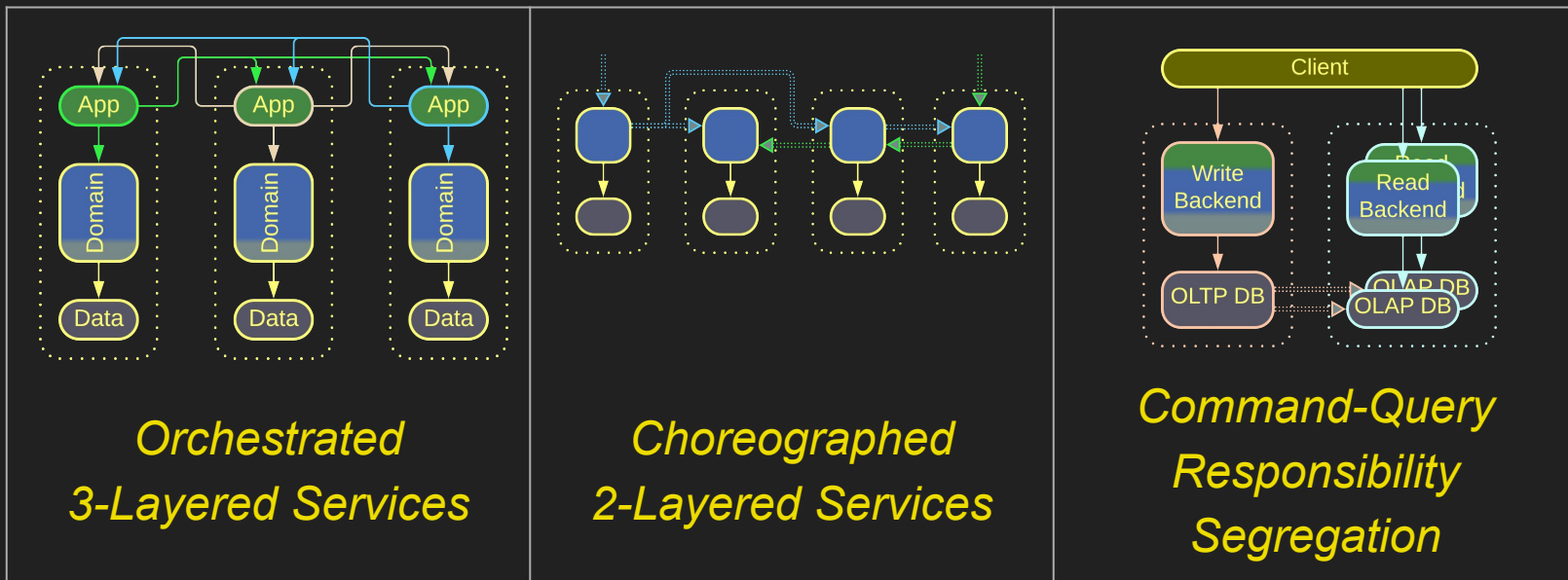
The components are color-coded

Layered Services

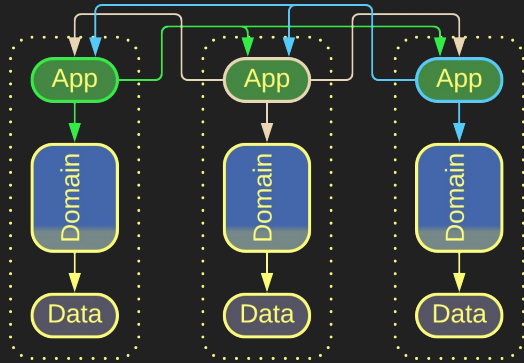
Each service is made of layers

Layered Services – overview

Layered Services may intercommunicate at different levels:



Layered Services – variants



*Orchestrated
3-Layered Services*

The most common orchestrated architecture is *3-Layered Services*. Each service contains *application*, *domain rules*, and *infrastructure* layers.

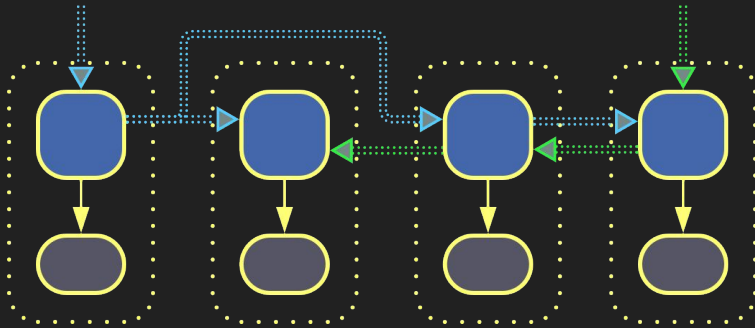
The *application*-level components may call each other, linking the services into a coherent whole.

Other layers remain isolated within their services.

Layering isolates interservice dependencies.

But it takes certain effort to implement and maintain.

Layered Services – variants



Choreographed 2-Layered Services

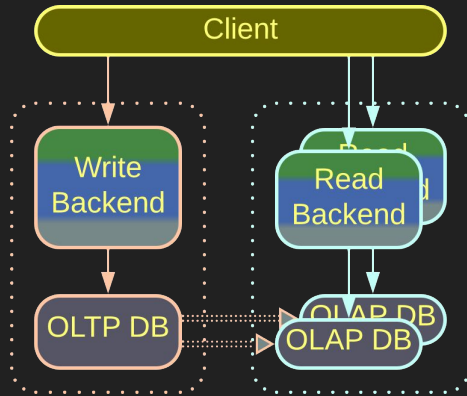
A choreographed system has no application layer. The use cases are encoded in the connections between the services which make a *Pipeline*.

In *2-Layered Services* each service stores its data in a private database for the sake of encapsulation.

Private databases make the services almost perfectly independent.

But they add to operational complexity.

Layered Services – variants



CQRS

When one service handles all write requests (*commands*) while another one is dedicated to reading (*queries*), that's *Command-Query Responsibility Segregation (CQRS)*.

The *command* and *query* services may have separate databases (*OLTP* and *OLAP*, respectively) and the *query* service may scale independently.

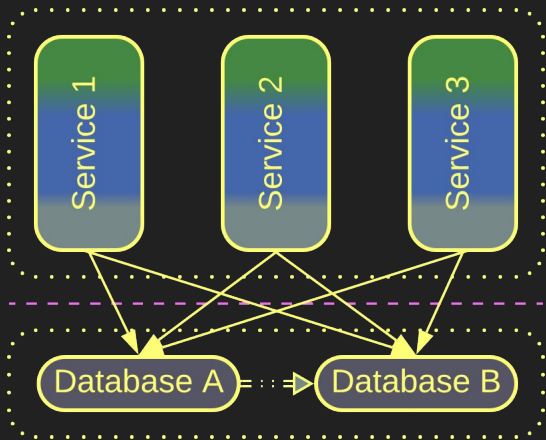
CQRS may simplify both commands and queries.

But in many cases it may be an overkill or a wrong abstraction.

Polyglot Persistence

Multiple storages

Polyglot Persistence – overview



Polyglot Persistence

Polyglot Persistence is found in high-load or low latency systems.

It may be too complicated and fragile for general use.

You can rely on dedicated storage technologies for different kinds of data or patterns of data access in your system.

The storages may be independent or one may derive its data from another. They may be private to a service or shared within the system.

Polyglot Persistence – trade-offs

Performance is fine-tuned for multiple access patterns

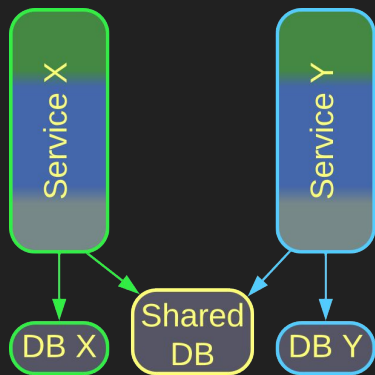
The load is distributed among the datastores in use

Multiple technologies need to be mastered

There is more operational complexity and more points of failure

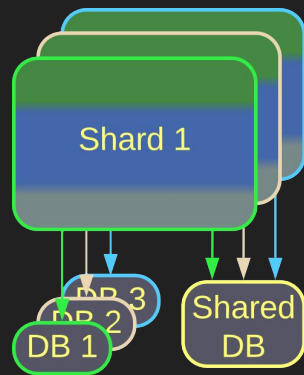
Data consistency becomes an issue

Polyglot Persistence – independent DBs



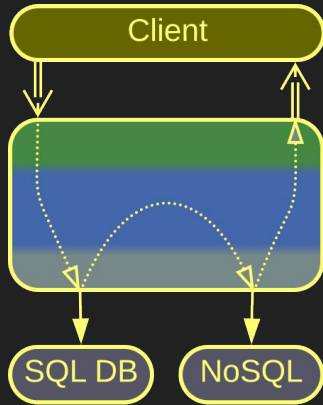
*Private and
Shared
Databases for
Services*

If *Shards* or *Services* become coupled through a small subset of the system's data, that subset can be extracted into a *Shared Database*, while the main dataset remains private.



*Private and
Shared
Databases for
Shards*

Polyglot Persistence – independent DBs

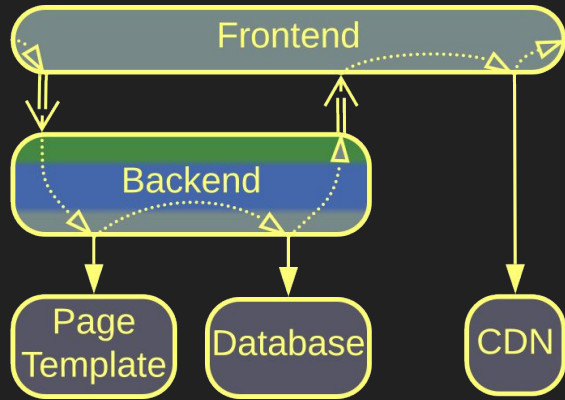


*Specialized
Databases*

There are many database engine technologies, each optimized for a small number of use cases and data types.

If you need top-grade performance, you can hand-pick databases that best fit the structure and access patterns for subsets of of your data.

Polyglot Persistence – independent DBs

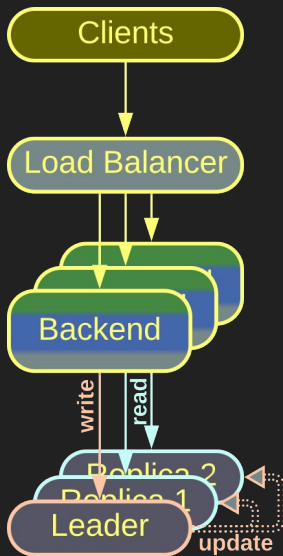


*Data File and
Content Delivery
Network*

Some data types, such as web page templates, images, or videos, are best served from files.

A *Content Delivery Network (CDN)* is a distributed file storage that replicates web content over the globe so that every user downloads it from the nearest data center.

Polyglot Persistence – derived DBs



Read-heavy systems benefit from running multiple read-only *replicas* of the database.

One instance of the database, called *leader*, accepts all write requests and propagates changes to other *replicas* used in read-only queries.

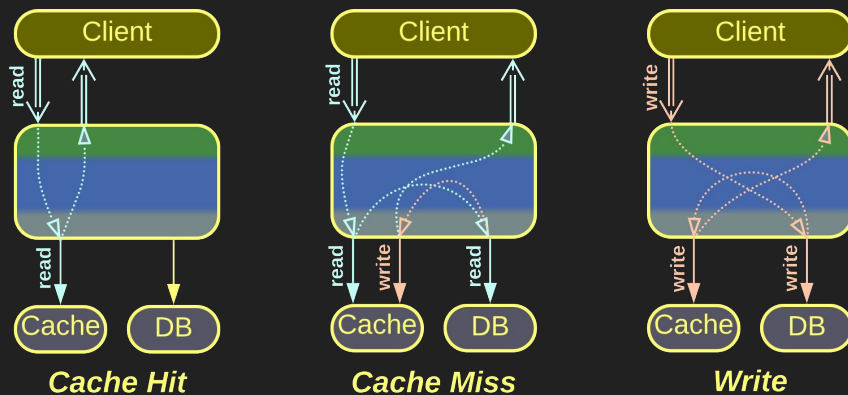
If the *leader* becomes unavailable, any other *replica* is promoted to the *leader* role.

Read-Only Replicas

Polyglot Persistence – derived DBs

A *Database Cache* stores the most recent database queries and their results in memory to bypass the main database when a known query is repeated.

The hard part is keeping the data in the *Cache* and the database in sync.

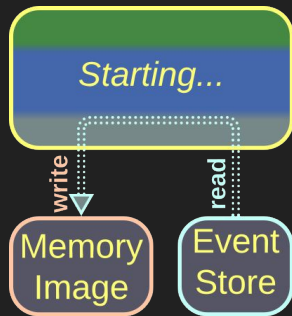


Cache-Aside, Database Cache

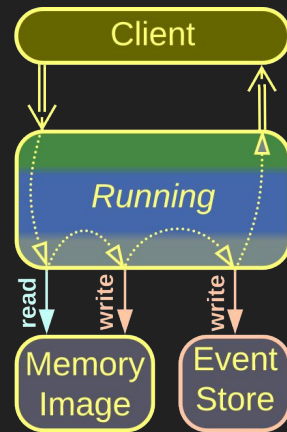
Polyglot Persistence – derived DBs

A *Memory Image* aggregates events into an up-to-date snapshot of the (sub-)system's state.

It is queryable, unlike the *event store*.

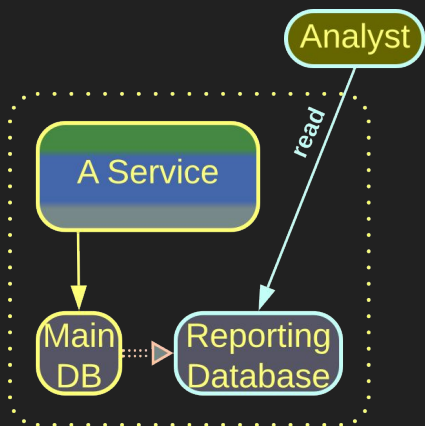


Memory Image
– starting



Memory Image
– running

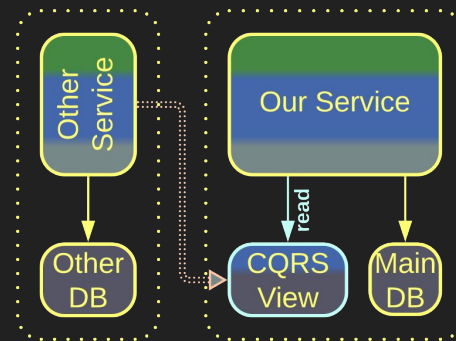
Polyglot Persistence – derived DBs



Reporting Database

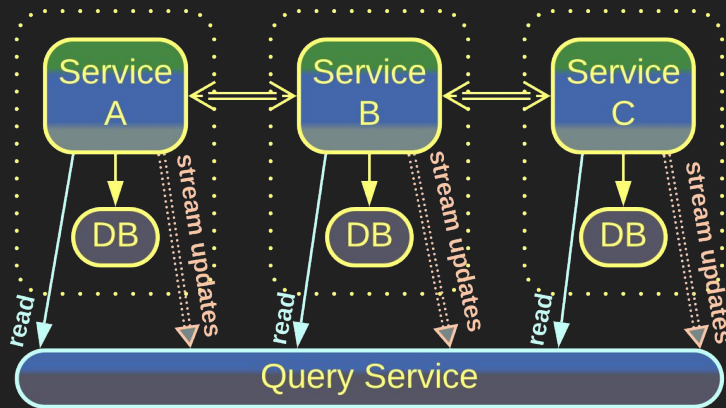
Here we have dedicated *OLTP* (*commands*) and *OLAP* (*queries*) databases.

- A *Reporting Database* exposes the dataset of this service to external queries.
- A *CQRS View* aggregates events from another service into a queryable state.



CQRS View

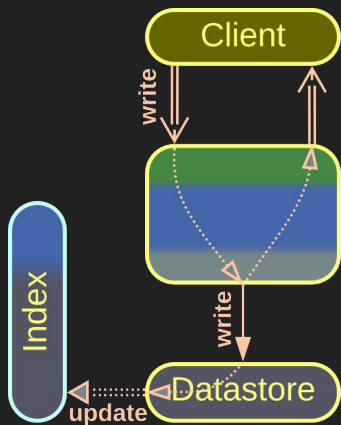
Polyglot Persistence – derived DBs



Query Service

A *Query Service* is a shared *CQRS View*. It aggregates events from multiple services to provide a consistent, queryable representation of the state of the whole system.

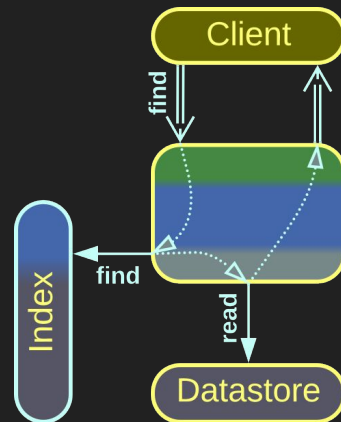
Polyglot Persistence – derived DBs



External Index
– update

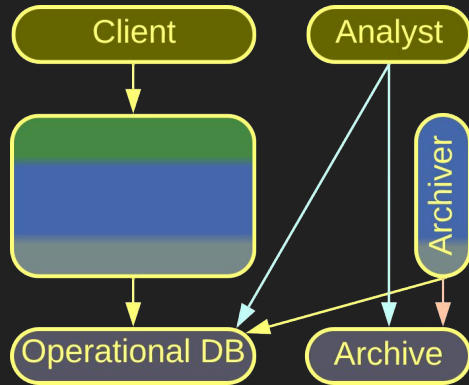
An *External Index* is a query engine dedicated to an uncommon technology, such as NLP or geospatial data.

You can use it to get a list of all the rows in your main database that match a special kind of query not supported by your primary database.



External Index
– find

Polyglot Persistence – derived DBs



Data Archiving

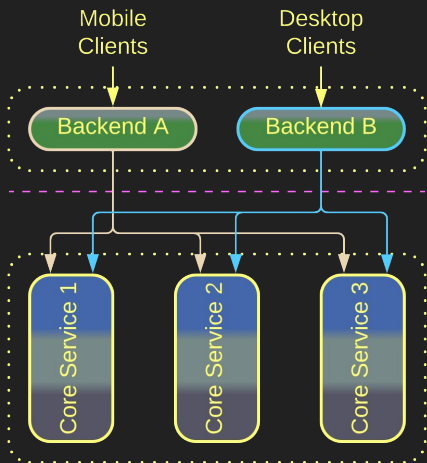
Keeping old data which is not going ever to change together with the most recent records bloats and slows down your main OLTP database.

Therefore you should periodically move the old data to a read-only archive OLAP storage.

Backends for Frontends

Each kind of client gets a dedicated service

BFF – overview



Backends for Frontends

BFF is useful with dissimilar client protocols, UIs or workflows.

Its use with closely related clients asks for trouble.

If you clients differ in their protocols and/or workflows, you can dedicate a component (*Backend for Frontend, BFF*) to each kind of client.

The *BFFs* may act as *Proxies*, *Orchestrators*, or their combinations.

BFF – trade-offs

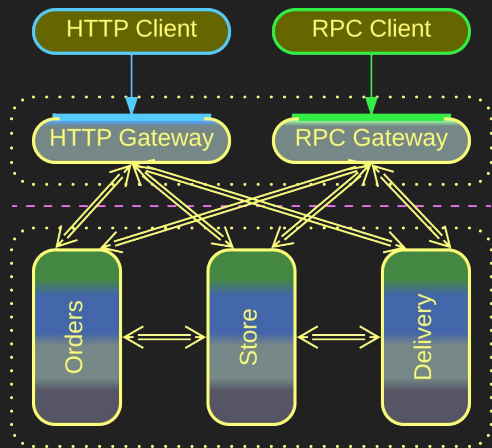
Clients and the teams that support them become independent

Each of the multiple BFFs tends to be much simpler than a universal layer

It's hard to share code between BFFs

BFFs increase operational complexity

BFF – variants



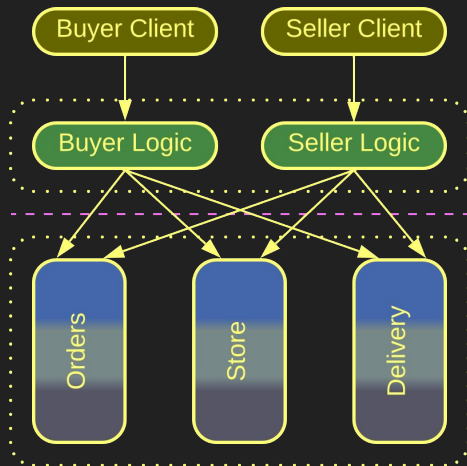
Proxies

Multiple *Proxies* are made use of to support clients that differ in their protocols but not workflows.

An example is accessing the system from the internal network and via a VPN.

Proxies are often available off the shelf and don't require much effort to set up.

BFF – variants



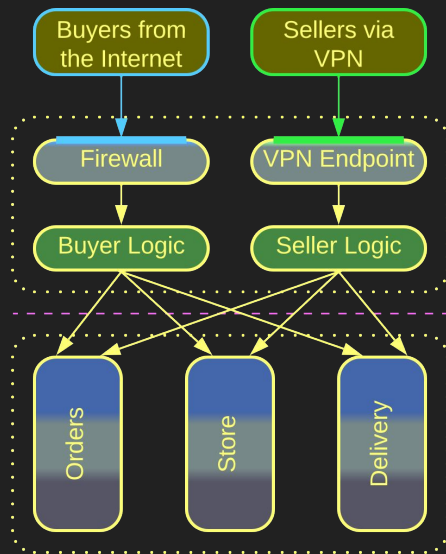
Orchestrators

BFF Orchestrators deal with workflows but not protocols.

For example, an online store may use different *Orchestrators* for buyers and sellers.

Each *Orchestrator* is likely to remain relatively small and simple thanks to the separation of concerns.

BFF – variants

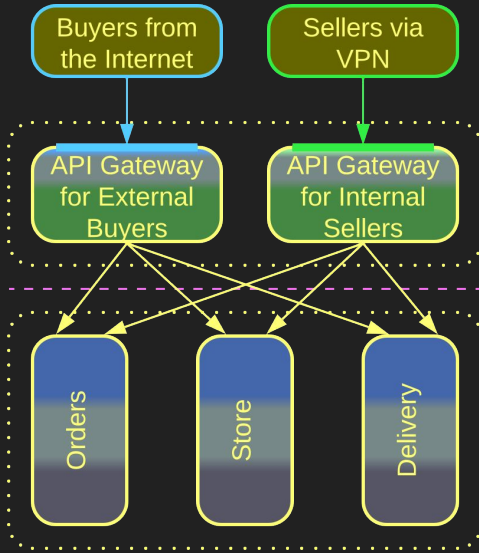


*Proxy +
Orchestrator pairs*

When the clients differ in both protocol or security, and their workflows, *Proxy + Orchestrator* pairs are used.

This also allows for reusing *Proxies* and/or *Orchestrators* in different combinations.

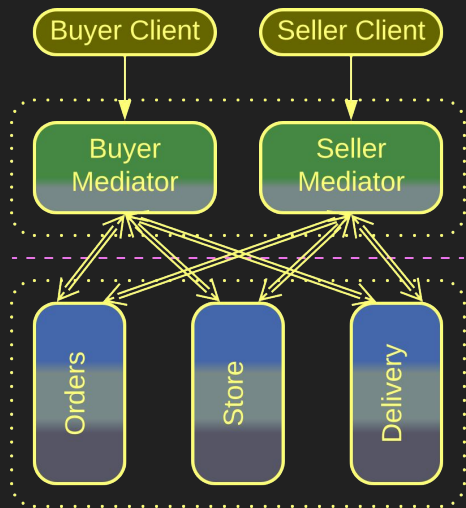
BFF – variants



API Gateways

An *API Gateway* implements aspects of both a *Proxy* and *Orchestrator*. Therefore it can be used as a more convenient alternative to a *Proxy* + *Orchestrator* pair.

BFF – variants



Event Mediators

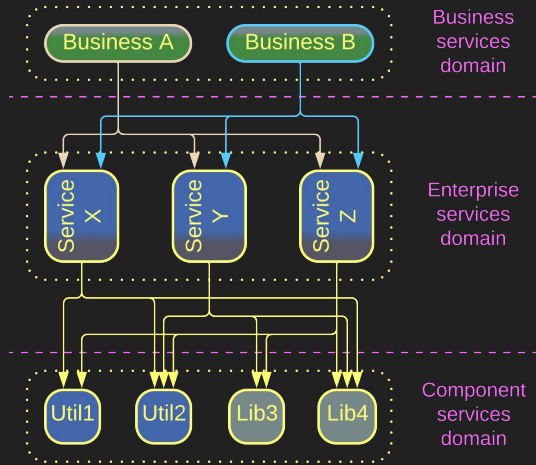
An *Event Mediator* is an *orchestrating Middleware*. There are reports of using an *Event Mediator* per kind of system's clients to achieve the following benefits:

- Smaller and more cohesive orchestration logic for each kind of client.
- If one *Event Mediator* malfunctions, the system remains available to other kinds of clients.

Service-Oriented Architecture

Each layer is made of services

SOA – overview



Service-Oriented Architecture (SOA) first splits a system into *Layers*, then divides each layer into *Services*. This is similar to how OOP/OOD treats a complex project.

The resulting architecture addresses the complexity of the domain, but is itself quite complicated.

Service-Oriented Architecture

SOA suits huge projects, often with heterogeneous hardware.

But don't expect fast-paced development or low latency.

SOA – trade-offs

Supports huge codebases with many development teams and technologies

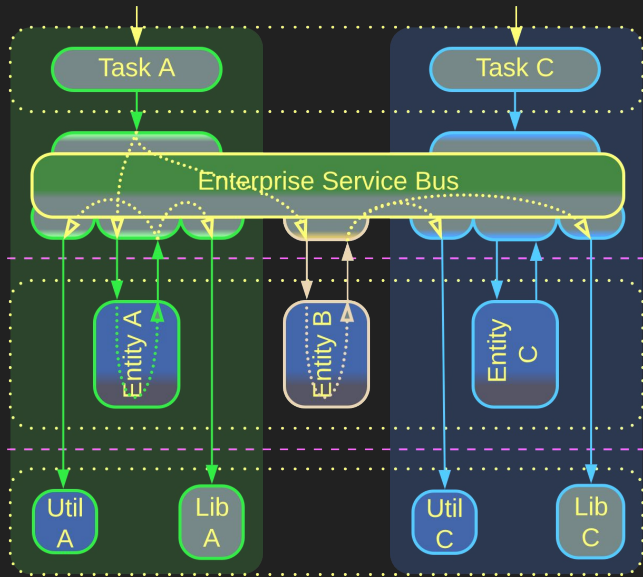
Provides fine-grained scalability and deployment to dedicated hardware

The teams are interdependent, which slows down development

The excessive distribution makes the system slow, fragile, and hard to debug

The system is hard to maintain or test because it contains many tightly interacting components

SOA – examples



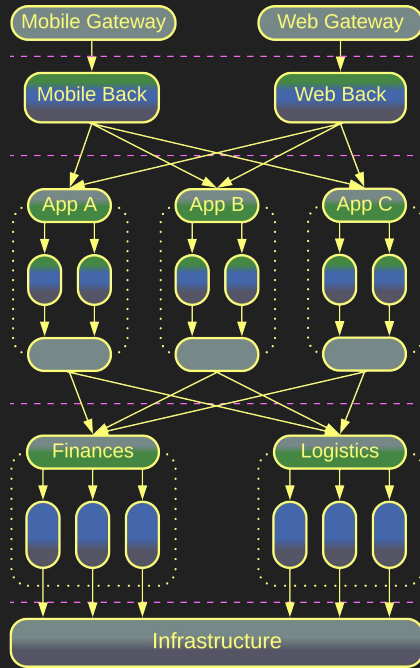
Enterprise SOA

Enterprises used to enlarge by acquiring smaller companies, each with its own homegrown software system that had to be integrated into the enterprise's architecture.

The integration was mediated by an *Enterprise Service Bus (ESB)* responsible for both protocol translation and orchestration.

However, the resulting complexity of the *ESB* made everyone hate it.

SOA – examples

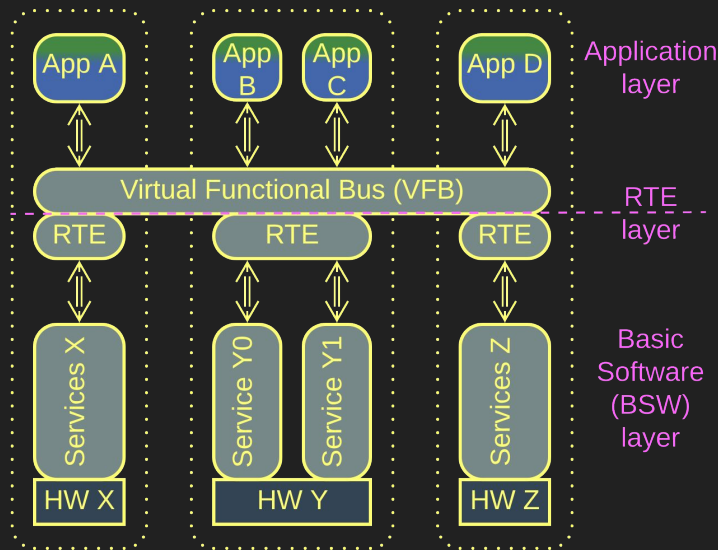


Domain-Oriented Microservice Architecture (DOMA) is made up of *Cells* (domains, groups of related services) instead of plain services. It relies on *Plugins* for cross-domain customization.

DOMA seems to be a flexible real-world architecture suitable for huge projects.

Domain-Oriented Microservice Architecture

SOA – examples



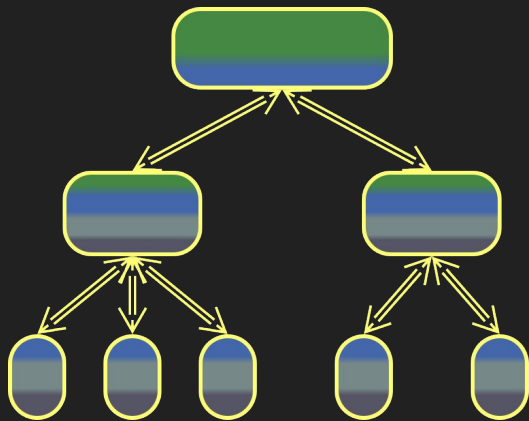
Automotive SOA

AUTOSAR Classic from the automotive industry is marketed as a SOA (with its *Virtual Function Bus* matching *ESB*) but, on closer inspection, it looks like a *Microkernel*.

Hierarchy

A recursive structure

Hierarchy – overview



Hierarchy

Hierarchy distributes complexity over a tree-like structure. It is straightforward, convenient, and agile, but few domains are truly hierarchical.

A Hierarchy can be built:

- *Top-Down (Orchestrator of Orchestrators)*
- *Bottom-Up (Bus of Buses)*
- *In-Depth (Cell-Based Architecture)*

Hierarchy shines with large projects and/or physical devices (IoT).

Making a good hierarchical decomposition may be hard or impossible.

Hierarchy – trade-offs

It's good in decoupling logic

Multiple development teams are supported

Low-level components are easy to add or replace

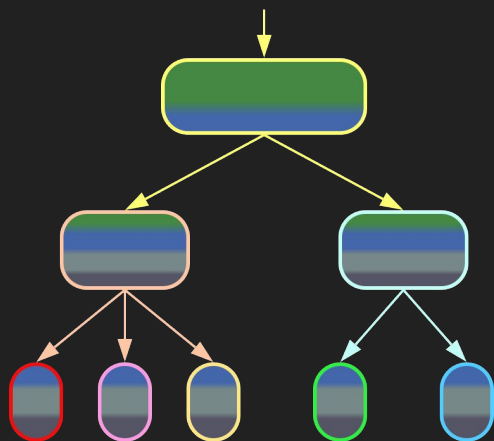
The system is quite fault-tolerant as its parts may act independently

System-wide use cases are slow and hard to debug

Hierarchy has high operational complexity

It also requires much planning or trial and error to discover good interfaces

Hierarchy – variants



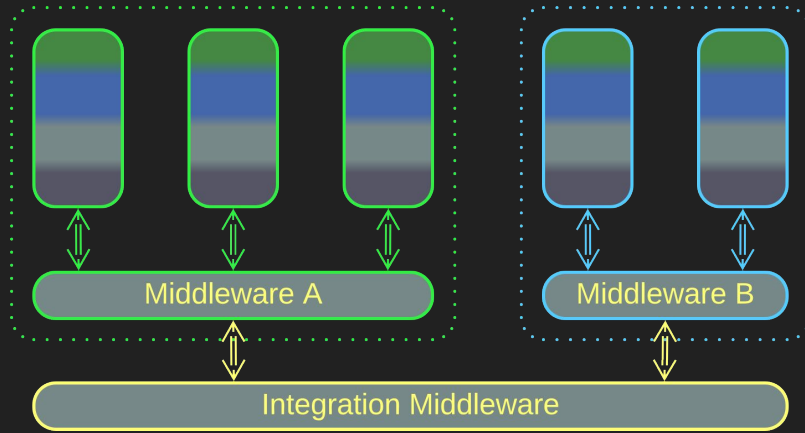
Top-Down Hierarchy

A *Top-Down Hierarchy (Orchestrator of Orchestrators)* builds a tree whose root deals with high-level commands or notifications while its leaves are responsible for concrete actions.

Examples include:

- Product filters in web marketplaces.
- Automation of industrial complexes (IIoT).
- Hierarchical UIs (*PAC* and *HMVC*).

Hierarchy – variants



Bottom-Up Hierarchy

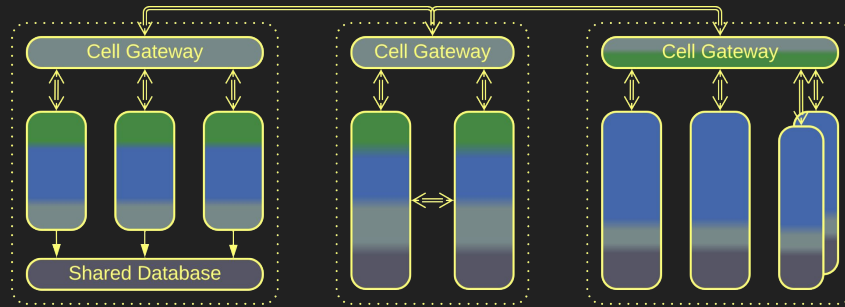
A Bottom-Up Hierarchy (Bus of Buses) interconnects several networks that differ in their internal protocols.

This is what the Internet is.

Hierarchy – variants

An *In-Depth Hierarchy* relies on a recursive subdivision of elements. A common example is the WSO2 definition of a *Cell* that is a subsystem made of services which itself takes the place of a single service in a larger system:

- *Cell-Based Architecture* is a single layer of *Cells*.
- *Domain-Oriented Microservice Architecture* is a *SOA* made of *Cells*.



In-Depth Hierarchy, Cell-Based Architecture

Conclusion

Subdividing a component

If a component conjoins loosely coupled segments, subdividing it may:

- Yield smaller and better structured code thanks to the separation of concerns.
- Improve performance through the careful selection of technologies.
- Improve fault tolerance if the newly created parts of the system may act independently.

However, it also:

- Increases operational complexity and latency.
- Makes sharing code inconvenient.

Links

This was a brief summary of part 4 of *Architectural Metapatterns*:

- read it online at metapatterns.io
- download the eBook from leanpub.com/metapatterns

The diagrams and the ODT source file are available under the CC BY license.

If you like this book, please tell your friends about it.
I have no way to promote it on my own.